CS3103 Assignment 4: LiveRecall System

Group 14: Muhammad Bin Hamzah (A0272938N), Ong Qi An (A0281202W), Tay Cole Lin (A0210841U), Nicholas Kee (A0308575W)
Repo: https://github.com/blobfish465/LiveRecallSystem

1. Problem

A caption source emits a lecture as numbered segments, each sent as provisional revisions and then one final.

Our server, `solution_server.py`, relays this stream to many clients and must ensure that:

1. no client is shown an older revision after a newer one;
2. every final reaches every client, including those that drop out or join late;
3. captions stay fresh: caption age = t_receive − source_ts ≤ 500 ms;
4. memory stays bounded when clients read slowly or sit on lossy or narrow links.

The server is asyncio Python (3.8+) with `websockets` as its only runtime dependency. We also built a browser client and test tools that drive the unmodified instructor harness.

2. Architecture and protocol

The code has three layers (Figure 1). `CaptionState` holds the revision rules and the bounded history of finals. `ClientChannel` gives each client a bounded queue and a sender task, and holds all the backpressure policy. `solution_server.py` connects them to WebSocket connections and sets two transport options per connection. The first two layers never touch a socket. 84 of our 154 pytest tests drive them directly through fake send and ping functions, 24 of them with an injected clock.

Protocol. The source sends `SOURCE_HELLO`, then `CAPTION_UPDATE`, and optionally `SOURCE_DONE`, which is never forwarded. A client sends `CLIENT_HELLO`. It receives one `HISTORY` containing the finals it does not already hold, oldest first, followed by live `CAPTION_UPDATE`s. We add no message types. Lag detection uses standard WebSocket ping/pong control frames, which every client library answers on its own.

Three design rules shape everything else:

1. **The source never waits for a client.** Each update is applied to the state and then handed to every channel with `offer()`, which never awaits. It queues the update or, for a lagging client, overwrites that client's unsent provisional of the same segment. The starter awaited every send, so the slowest client set everyone's pace.
2. **Serialise once.** Each accepted update becomes one JSON string with only the seven protocol fields. That string is shared by all clients and spliced unchanged into `HISTORY`. Unknown fields and rejected updates never reach a client.
3. **Snapshot and subscribe in one step.** On `CLIENT_HELLO`, the server builds `HISTORY` and registers the client for live updates with no `await` in between. asyncio switches tasks only at an `await`, so no update can fall between the two: anything applied before the snapshot is in `HISTORY`, and anything after it is queued behind `HISTORY`. Each final therefore arrives exactly once, and in order.

3. Revision and recovery semantics

Revision rule. The server accepts an update only if all three hold:

- its revision is higher than the one held for that segment;

- the segment is not final yet (finals are durable);
- the segment lies above the eviction floor.

Rejected updates are never forwarded. Nothing assumes a revision count (tests cover 1, 2, 3 and 8).

Bounded state. The history of finals is limited to 90 s **and** 6000 items, which still covers 60 s at 100 segments per second. Each `HISTORY` reply is capped at 768 KiB, below the 1 MiB message limit of websockets clients; if the window is larger, the oldest finals are left out. At most 64 segments may be in progress at once. Evicted segments are remembered as one integer, so late stragglers are still rejected. Per-client state is bounded as well (Section 5). As the updated requirement 8 allows, a client away for longer than the retained window is not guaranteed its oldest finals.

Ordering. Each channel sends in queue order over TCP. Coalescing (Section 5) replaces a queued provisional in place, so nothing older overtakes something newer. Snapshot-and-subscribe (Section 2) makes each final arrive exactly once across `HISTORY` and live delivery. The browser applies the same rules (client/store.js), so even a repeated message is never displayed twice.

Recovery. A client reports the newest *final* it holds, never a provisional. If it reported a provisional of segment *s*, the server may think the client has the final segment and would never send *s*'s final.

- A fresh client (`last_segment = 0`) receives the finals of the last `history_seconds`, clamped to 60 s.
- A resuming client receives every retained final after its reported state, ignoring `history_seconds`, so a client that was away slightly longer than it asked for still ends up complete.

Neither gets the in-progress segment's provisional on connect. The server does not know which provisional the client already holds, so sending one could duplicate it; the next revision arrives within about 0.35 s anyway.

4. Browser client and usability

The student client is a static page (`client/`) that updates one row per segment in place and answers "What did I miss?" (15, 30 or 60 s) from finals stored locally, so it works while disconnected. It reconnects by itself and resumes from its newest final. Details and screenshots are in Appendix E.

5. Backpressure and overload policy

The problem**.** During T3 our per-client queue stayed empty while slow readers fell seconds behind. `ws.send()` returns as soon as bytes reach the socket buffer, so the backlog builds in kernel buffers and in the client library's 16-message read-ahead, where the server cannot see or edit it (Appendix D, Figure 2).

Detection. A full read-ahead also stops the client answering pings. Each channel pings at most once per second, and a pong later than 1 s marks the client as lagging. The kernel's TCP RTT (`TCP_INFO`) then gives the cause, because the client's kernel acknowledges data even when its app is not reading. Below 50 ms the client is a **slow reader**. At 50 ms or more it is a **slow link**, confirmed by two late pongs in a row, and a pong pending longer than max(6 s, 4 × RTT) reclassifies it as a slow reader.

Lag mode.

- Finals are sent as soon as they arrive and are never dropped.
- A new provisional overwrites the unsent provisional of its segment in place (coalescing), so the queue holds at most one provisional per segment.

- A provisional is safe to drop once a newer revision or the final of its segment is queued, because the client would replace it on arrival anyway.
- Each message keeps its own `source_ts`, and HISTORY is only ever sent after a `CLIENT_HELLO`.

Recovery and bounds. Catching up can only be tested by retrying full delivery, after 60 to 90 s of fast pongs for a reader and 5 to 15 s for a link, doubled on each relapse up to 120 s. A queue over 64 entries, a caption queued over 10 s, or a reader's ping unanswered for 10 s closes the client with code 1013, and it recovers from HISTORY on reconnect. A lag memory of up to 4096 client ids makes a reconnect within 120 s resume lag mode. Send buffers are capped at 512 B and TCP segments at 1400 B, so the backlog stays in our queue and large HISTORY replies survive the 2 Mbit/s shaper.

Appendix D, Figure 3 shows the lag states. The baseline, `LIVERECALL_NO_COALESCE=1`, turns all of this off (plain FIFO).

6. Public experiments

Setup (unless stated): Ubuntu 26.04 VM (Apple Silicon, 2 vCPU), Python 3.14.4, websockets 16.1.1, final code, unmodified harness, median of 3, timer drift +0 to +1%.

Table 1. T1 to T4 and late join, median of 3. Every run had 0 duplicates, 0 stale messages and 0 connection errors.

Metric	T1 (20 clients)	T2 (100 clients)	T3 (10% slow)	T4 (20% drop 5 s)	Late join
Final completeness (%)	100.00	100.00	100.00	100.00	100.00
Normal update coverage (%)	100.00	100.00	94.56	98.33	92.25
P50 / P95 / P99 (ms)	2.4 / 5.1 / 10.4	5.9 / 17.7 / 60.7	5.8 / 22.0 / 7524.3	5.7 / 14.4 / 76.6	5.3 / 11.9 / 16.9
Fresh (%)	100.00	100.00	96.38	100.00	100.00
Fresh finals (%)	100.00	100.00	93.16	98.68	92.65
Recovery P50 / P95 (ms)	–	–	–	4.2 / 7.4	1.0 / 2.6
Duplicates/Stale	0/0	0/0	0/0	0/0	0/0

- **Slow readers do not hurt normal ones.** T3 normal clients kept a P95 of 12.6 ms. Slow readers sat at 9.6 s, mostly the 8s to drain a 16-message read-ahead at 2 messages per second.
- **Coverage below 100% is intended.** T3 slow readers get coalesced provisionals, and T4 and late clients recover missed segments as finals only.

T5 (full table in Appendix B). Every profile kept 100% of finals. Under public delay_loss, P50 / P95 / P99 was 262 / 675 / 894 ms with 99.89% coverage. With 3% loss alone, P50 stayed at 6.9 ms while P99 rose to 220 ms.

Loss hurts only the tail. TCP delivers in order, so only captions queued behind a lost segment wait for its retransmission. The plain FIFO shows the same shape (P95 659 ms), so this tail comes from TCP itself.

7. Before/after comparisons

Each comparison runs the same test against the plain FIFO (LIVERECALL_NO_COALESCE=1) and our lag-aware policy. Table B4 (Appendix B.4) summarises the results.

Slow readers (T3). *Before:* under the FIFO, slow readers fall 28 s behind (P95) and are still reading old captions when the test ends, so they miss its last finals: 95.13% in every run, just above the 95% line. *Change:* ours detects the lag from late pongs and coalesces those clients' provisionals, keeping only the newest per segment and every final. *After:* slow readers stay at the ~9 s floor set by the client library's 16-message read-ahead (P95 9.6 s) and receive every final (100% in every run). Normal clients are unaffected (P95 11.5 vs 12.3 ms).

Constrained bandwidth (500 clients, 2 Mbit/s). *Before:* both configurations saturate the link (1.6–1.7 Mbit/s average), so not every revision can be delivered. The FIFO queued about a second of everything, delivered no finals fresh, and lost finals in every run (98.5–99.1%). *Change:* ours classifies the clients as slow-link lag from their RTT, coalesces their provisionals, sends finals at once, and caps its pings. *After:* ours kept 99.97–100% of finals, cut P50 by about 40% (1103 → 671 ms), and delivered about a third of all finals fresh (≤ 500 ms). The cost is coverage: lagging clients get only the newest provisional per segment (57.5% vs 98.3%).

Slow readers over 5 minutes. *Before:* under the FIFO, slow readers fell further behind the longer the lecture ran: 47 s (P95) after 5 minutes, against 28 s in the 60 s T3. Completeness dropped to 98.38%, with 20 connection errors. *Change:*the same coalescing as in T3, run for 5 minutes. *After:* ours held them at the ~9 s read-ahead floor (P95 9.0 s), with 99.90% completeness, no connection errors, and flat server memory (37.1–37.3 MB, Figure 6).

8. Failures and trade-offs

Table 3. Policy failures and fixes. Results on the final code, median of 3. Rows 4–6 from hidden_sim.py (row 1 failure data observed on Day 3, Mininet VM).

Failure	Cause	Fix	Result
2 Mbit/s, 500 clients: 94.3% finals, P95 9.3 s	ping per send outweighed 17-byte captions, stop-and-wait lag mode	ping budget, finals sent at once, RTT cause, 512 B buffers	99.97–100%, P95 1.2 s

Public T5: coverage 93.5%	one retransmission read as a slow link	2 late pongs confirm a slow link; a relapse within 120 s is believed at once	99.89%
T3: 99.83% finals	jitter let a slow reader retry after 30 s	reader jitter only lengthens the wait	100%
Readers at 0.7 s/msg: 98.2%	reconnect restarted full delivery	lag memory	100%
2 Mbit/s + slow readers: 99.78%	readers on a congested link classed as link lag	reader reclassification	99.75%
2 Mbit/s late joiners: 80%	64 KiB HISTORY segment exceeds the token bucket filter burst	MSS 1400 B	100%
Delayed links regressed: delay_100ms P95 254 → 393 ms	Manual listening socket (for the 1400 B segment cap) had proto 0, so asyncio left Nagle's algorithm on	Create it with IPPROTO_TCP	P95 254 ms

Trade-offs. Each one gives up something graded more lightly to maximise final completeness which, below 99%, caps the hidden-performance mark.

- **Slower return to full delivery.** A 10 s retry wait cost T3 finals (98.3%), so readers wait 60 to 90 s. One who speeds up gets every revision again after about 70 s.
- **Coverage for finals.** Coverage fell to 58% on 2 Mbit/s (FIFO 98%) and to 80% on ~300 ms links with 3% loss (FIFO 99%), with every due final delivered in both.
- **Ping overhead.** P95 at 1000 clients was 61 ms (FIFO 55 ms). When the stream only just fits a 2 Mbit/s link (300 clients), the FIFO stays fresher: P95 343 vs 875 ms.
- **Remembered lag.** A client that was lagging and reconnects resumes coalescing for up to 120 s, so one whose link has improved may see fewer provisionals at first. Finals are unaffected.

9. Limitations

- **9 s floor** for **slow readers:** the client library's 16-message read-ahead is out of the server's reach.
- **Absolute RTT threshold:** on high-latency links slow readers can be mistaken for a slow link.
- **Older clients:** (Python 3.8, 1 run) websockets 13.1 clients read ahead 32 messages, so T3 falls to 95.5% (FIFO 95.1%).
- **End-of-run tail:** a slow reader may not receive the last few finals before a short grace period ends: 99.90% over 5 minutes, 99.75% on 2 Mbit/s with slow readers (Appendix C).

10. Future work and FYP direction

Client acknowledgements would replace the ping inference, and QUIC would avoid TCP head-of-line blocking. As an FYP: adaptive caption delivery over congested campus Wi-Fi with on-device speech recognition.

References

[1] CS3103 Assignment 4 specification, LiveRecall System, and starter documents (START_HERE.md, PROTOCOL_EXAMPLES.md, NETWORKING_PRIMER.md, PUBLIC_TESTS.md).
[2] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, 2011.
[3] websockets 16 documentation, "Keepalive and latency" and "Memory and buffers," https://websockets.readthedocs.io
[4] Linux manual page tcp(7): TCP_INFO and TCP_NOTSENT_LOWAT.
[5] Linux manual pages tc-tbf(8) and tc-netem(8).
[6] W. Eddy (Ed.), "Transmission Control Protocol (TCP)," RFC 9293, IETF, 2022.

Appendix A: Reproducing the results

pip install -r requirements-dev.txt
python -m pytest -q
python check.py --full --runs 3 (Table 1: T1 to T4 and late join)
python check.py --stress --runs 3 (Table B2)
T5, for each profile in delay_loss, delay_100ms, loss_3pct, reorder (Table B1):
  sudo ./network_profiles/<profile>.sh lo
  python run_demo.py --clients 100 --duration 40 --runs 3 --output-stem t5_<profile>
  sudo ./network_profiles/reset.sh lo
T3 before/after (Table B4):
python run_demo.py --clients 100 --duration 60 --runs 1 --slow-percent 10 --slow-ms 500, alternating plain FIFO and ours, 3 times each
python bandwidth_test.py --apply --clients 500 --duration 60 --runs 3 (Table B4, Figure 5)
5-minute run (Table B4, Figure 6):
python run_demo.py --clients 100 --duration 300 --runs 1 --slow-percent 10 --slow-ms 500,
with python monitor_server.py --pid <server PID> --interval 1 --duration 300 (see README)
python convergence_test.py --runs 3 (Table B3)
python convergence_test.py --slow-until 90 --duration 230 --runs 1 (B.3, 90 s case)
python hidden_sim.py --runs 3 (Appendix C)
Plain-FIFO baseline: prefix run_demo.py or check.py with LIVERECALL_NO_COALESCE=1.
bandwidth_test.py, convergence_test.py and hidden_sim.py run both versions themselves.
Full step-by-step commands are in README.md.

Appendix B: Additional results

B.1 T5 network impairment

Table B1. 100 clients, 40 s, median of 3. Final completeness was 100% in every row.

Profile	P50 / P95 / P99 (ms)	Final completeness (%)	Normal update coverage (%)	Fresh (%)	Fresh finals (%)
delay_loss	262.1 / 674.7 / 894.4	100.00	99.89	90.69	90.45
delay_100ms	206.4 / 253.6 / 271.6	100.00	100.00	100.00	100.00

loss_3pct	6.9 / 145.7 / 219.8	100.00	99.95	100.00	100.00
reorder	48.4 / 54.4 / 59.1	100.00	100.00	100.00	100.00
delay_loss (plain FIFO)	256.3 / 659.4 / 1031.4	100.00	99.99	91.90	90.10

B.2 Stress

Table B2. Stress tests, 40 s, median of 3.

Scenario	Final completeness (%)	Coverage (%)	P50 / P95 / P99 (ms)	Normal P95 (ms)	Slow P95 (ms)	Server RSS max (MB)
s300 (300 clients)	100.00	100.00	14.6 / 26.2 / 29.8	26.2	–	50.5–51.9
s500 (500 clients)	100.00	100.00	19.7 / 34.4 / 90.1	34.4	–	65.9–66.8
s500slow (500, 10% slow)	100.00	95.10	18.1 / 834.8 / 7869.6	31.3	9113.5	66.0–66.9

Median of 3, 40 s. 300 clients, 500 clients, and 500 clients with 10% slow readers all reached 100% finals with no duplicate, stale or hopeless-close events. P95 caption age was 26.2 ms, 34.4 ms and 31.3 ms for normal clients (9.1 s for slow ones). Coverage was 100%, 100% and 95.1%, and server RSS peaked at 52, 67 and 67 MB. On 2 vCPU the load generator shares the machine with the server, so these are upper bounds on server latency.

B.3 Convergence after overload

Table B3. Convergence: 100 normal + 10 clients reading at 500 ms/message for the first 30 s of 120 s, then at full speed. Median of 3.

Metric	Plain FIFO (before)	Ours (after)
Final completeness (%)	100.0	100.0
Normal update coverage (%)	100.0	95.5
Normal clients P95 (ms)	10.7	11.5
Converging P95, whole run (ms)	11565	8851
Slow-phase P95 (ms)	13562	9882
Backlog at switch (ms)	14574	6068
Fresh after (s)	0.21	0.19
Full delivery after (s)	0.2	69.5

Settled P95 (ms)	8	9

When 10 slow readers sped up after 30 s, both servers had them fresh again within 0.2 s. Full revision delivery returned at once under the FIFO and after 70 s under ours, by design (Section 8). After a 90 s slow phase the FIFO's backlog had reached 44.6 s and 10 clients had been disconnected, while ours had 12 ms of backlog and no errors.

B.4 Before/after comparison test results

Table B4. Before/after test data. (Rows 1-2: median of 3, row 3: 1 run each). Slow-client P95 is the median of each run's P95 over the 10 slow readers (runs: FIFO 28.1 / 28.1 / 28.1 s, ours 9.6 / 9.7 / 9.6 s). Fresh = caption age ≤ 500 ms.

Test	Finals (plain FIFO → ours)	Key latency (plain FIFO → ours)	Other metric(s) (plain FIFO → ours)
T3 slow readers (100 clients, 10% slow)	95.13% → 100%	slow-client P95 28.1 → 9.6 s	overall P95 2255 → 15.5 ms; slow-client fresh 1.7 → 25.0%
Bandwidth 2 Mbit/s (500 clients)	98.6% → 100%	P50 1103 → 671 ms	fresh finals 0 → 34.3%; coverage 98.3 → 57.5%
Slow readers over 5 min (100 clients, 10% slow; 1 run each)	98.38% → 99.90%	slow-client P95 47.2 → 9.0 s	normal P95 9.1 → 11.2 ms; connection errors 20 → 0; RSS 37.4 → 37.3 MB max

Appendix C: Hidden-style conditions

Ubuntu 26.04 VM (aarch64, 2 vCPU, Python 3.14.4, websockets 16.1.1), unmodified harness, final code (0e8982e), median of 3 runs, timer drift +0–1%. Network profiles applied to the test port only. Each cell is “plain FIFO → ours”. Finals are counted against those due before the run ended.

Scenario	What it tests	Finals due (%)	Coverage (%)	P95 (ms)	Normal P95 (ms)	Slow P95 (s)	Fresh finals (%)
slow30	30% of clients slow at 500 ms/msg	85.81 → 100.00	85.5 → 83.7	21396 → 6316	12 → 12	28.1 → 9.6	70.0 → 79.6

slow250	10% slow at 250 ms/msg (~the full stream rate)	100.00 → 100.00	100.0 → 100.0	69 → 61	13 → 15	0.2 → 0.1	100.0 → 100.0
slow700	10% slow at 700 ms/msg (1.43 msg/s, just above the finals rate)	93.78 → 100.00	93.7 → 93.5	18 → 17	13 → 14	35.6 → 14.3	90.0 → 93.3
revs8_slow	8 revisions per segment (7.6 msg/s, 0.95 finals/s), 10% slow at 500 ms	92.64 → 100.00	92.7 → 91.9	14 → 14	12 → 13	41.3 → 10.6	90.0 → 95.7
bursty_slow	3 s bursts at 8x every 10 s (~11 finals/s inside a burst), 10% slow at 400 ms	100.00 → 100.00	94.6 → 95.1	17 → 2968	11 → 11	35.2 → 31.5	90.0 → 90.0
burst3x_slow	source 3x faster (12 msg/s, 4 finals/s), 10% readers at 150 ms (6.7 msg/s: finals yes, all no)	95.92 → 100.00	95.7 → 94.0	4169 → 15	13 → 14	24.5 → 4.2	90.1 → 97.3
bw1m_250	1 Mbit/s, 250 clients	100.00 → 100.00	98.2 → 55.8	1534 → 1254	1534 → 1254	n/a → n/a	0.0 → 48.8
bw2m_revs8	2 Mbit/s, 300 clients, 8 revisions per segment (provisional-heavy)	100.00 → 100.00	99.9 → 80.5	1093 → 1062	1093 → 1062	n/a → n/a	0.0 → 0.0
bw2m_slow	2 Mbit/s, 500 clients, 10% slow readers at 500 ms	95.27 → 99.75	93.5 → 57.5	3243 → 2427	1527 → 1238	27.6 → 10.3	0.0 → 32.5
bw2m_loss	2 Mbit/s + 20 ms delay + 1% loss, 500 clients	100.00 → 100.00	98.5 → 60.9	2152 → 1666	2152 → 1666	n/a → n/a	0.1 → 21.4
bw2m_lifted	2 Mbit/s for the first 30 s, then unlimited (catch-up), 500 clients, 90 s	100.00 → 100.00	99.8 → 78.3	1313 → 1016	1313 → 1016	n/a → n/a	68.8 → 79.9
bw2m_reconnect	2 Mbit/s, 300 clients, 20% drop at 20 s for 5 s, 20% join late at 15 s asking 60 s	100.00 → 100.00	93.3 → 91.5	343 → 875	343 → 875	n/a → n/a	92.3 → 25.9

mass_all	100% of clients drop at 15 s for 5 s (simultaneous reconnect)	100.00 → 100.00	91.5 → 91.7	12 → 15	12 → 15	n/a → n/a	92.1 → 92.1
mass_half_30s	50% drop at 15 s for 30 s	100.00 → 100.00	77.7 → 77.7	13 → 14	13 → 14	n/a → n/a	78.1 → 78.1
outage_75s	20% drop at 10 s for 75 s (beyond the 60 s they may ask for)	100.00 → 100.00	84.6 → 84.6	12 → 13	12 → 13	n/a → n/a	84.7 → 84.7
reconnect_loss	20% drop for 5 s on the public T5 link (120+-30 ms, 3% loss)	100.00 → 100.00	97.8 → 97.7	743 → 688	743 → 688	n/a → n/a	88.3 → 86.2
flap	20% of clients drop abruptly every ~8 s and stay down 1 s	100.00 → 100.00	97.1 → 97.1	14 → 15	14 → 15	n/a → n/a	98.2 → 98.1
flap_slow	the 10% slow readers also drop every ~10 s (exercises lag memory)	99.70 → 99.93	94.9 → 94.2	454 → 16	13 → 15	7.8 → 7.6	90.4 → 94.1
late_long_text	unique text; 20% drop at 70 s for 5 s, 20% join at 55 s asking 60 s (large HISTORY), 100 s	100.00 → 100.00	87.9 → 87.9	13 → 14	13 → 14	n/a → n/a	88.1 → 88.1
bw2m_late_long	2 Mbit/s, 300 clients, unique text; 20% drop at 70 s for 5 s, 20% join at 55 s asking 60 s, 100 s	100.00 → 100.00	87.8 → 84.0	575 → 867	575 → 867	n/a → n/a	80.5 → 54.3
delay300	300+-100 ms each way (RTT ~600 ms)	100.00 → 100.00	100.0 → 99.9	850 → 846	850 → 846	n/a → n/a	12.2 → 13.6
delay300_loss	300+-100 ms each way + 3% loss	100.00 → 100.00	99.0 → 80.2	1613 → 1598	1613 → 1598	n/a → n/a	11.7 → 11.2
delay300_slow	300+-100 ms each way + 10% slow readers at 500 ms	95.28 → 100.00	95.0 → 94.6	2084 → 968	851 → 845	27.6 → 11.5	12.9 → 17.1

scale1000	1000 clients over 4 load processes	100.00 → 100.00	100.0 → 100.0	55 → 61	55 → 61	n/a → n/a	100.0 → 100.0
scale1500_slow	1500 clients over 6 load processes, 10% slow at 500 ms	95.42 → 100.00	95.3 → 95.2	1833 → 1522	67 → 67	18.6 → 9.9	90.0 → 90.0

On simulated hidden tests that test a variety of stress points, our final program delivered 100% of due finals in 23 out of 25 scenarios. In contrast, plain FIFO was 17 out of 25 scenarios. The two that fell short in our version still stay above the 99% cap:

- bw2m_slow
  - 99.75% (Runs 99.76%, 99.75%, 99.68%)
  - Only slow readers missed finals, and only the last ~10 segments before the run ended (segments 64–73 of 73 due). On a saturated link, a slow reader's finals queue behind its read-ahead and the shaped link, so the last few were still in transit when the test stopped. Every normal client received every final.
- flap_slow
  - 99.93% (Runs 100%, 99.92%, 99.93%)
  - The shortfall is 2–3 finals per run, all in the last two segments, for slow readers that had just reconnected. They had not yet recovered them from HISTORY when the run ended. Normal clients were unaffected.

Appendix D: Figures

Figure 1: Architecture

The caption source sends updates to `CaptionState`, which rejects stale and duplicate revisions and keeps the bounded history of finals. The source handler then calls `offer()` on one `ClientChannel` per client without awaiting any send. Each channel's sender task delivers HISTORY first and live updates after it (Section 2).

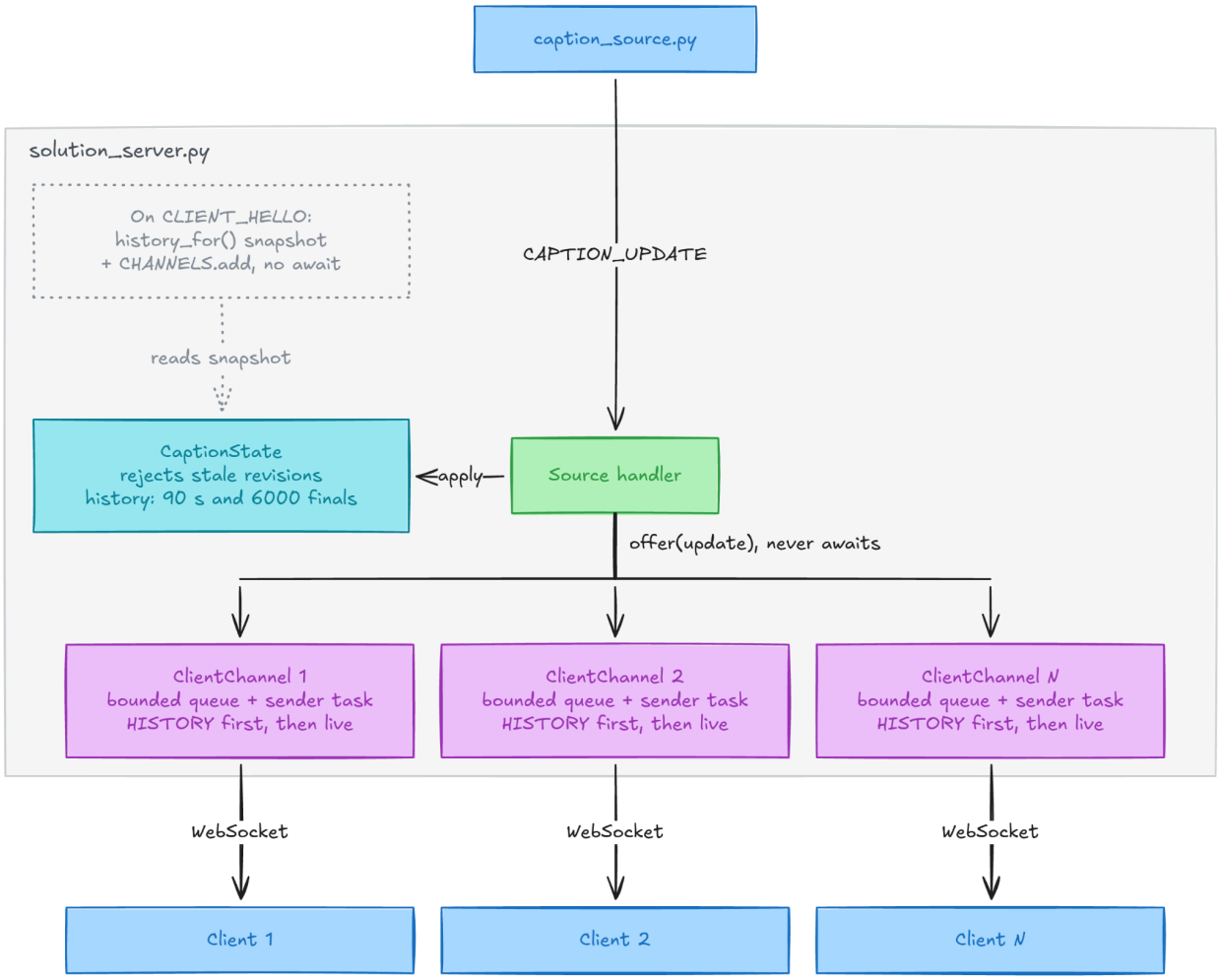


Figure 2: Where a slow reader's backlog sits

The path from the server's per-client queue through the kernel send buffer, the link, the client's kernel and the client library's 16-message read-ahead to the client application. The server can only see and edit its own queue, which stays empty while the backlog builds further along (Section 5).

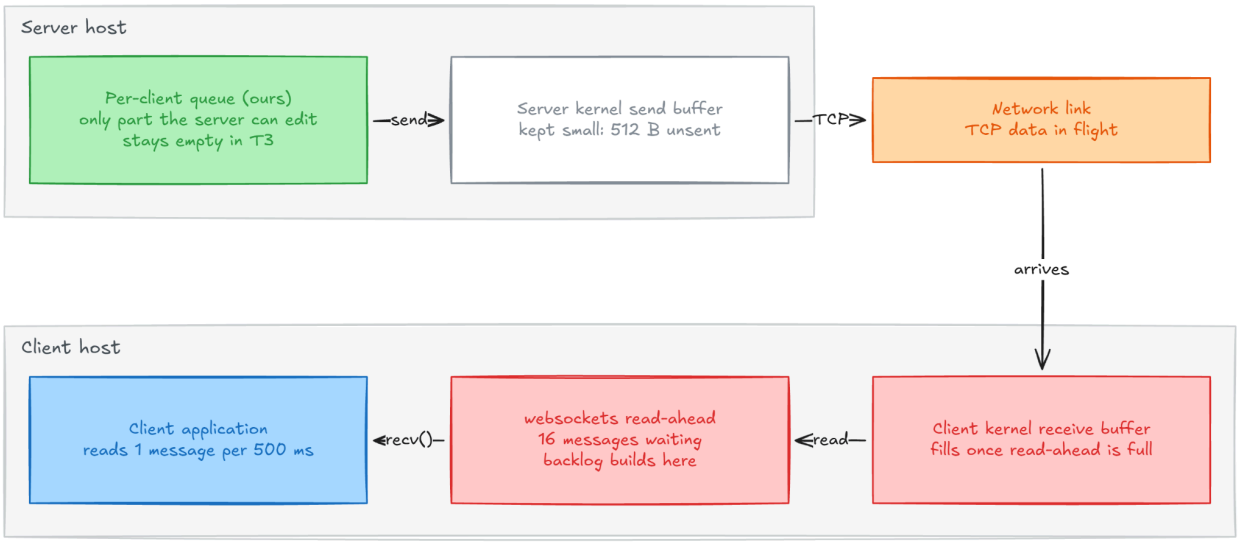


Figure 3: Lag state diagram

A channel starts in full delivery. A late pong moves it to lag mode as a slow reader or, after confirmation, a slow link, and a long-pending pong reclassifies a slow link as a slow reader. Lag mode returns to full delivery after the recovery wait, or ends in a hopeless close, after which the client reconnects, recovers from HISTORY and resumes lag mode through the lag memory (Section 5).

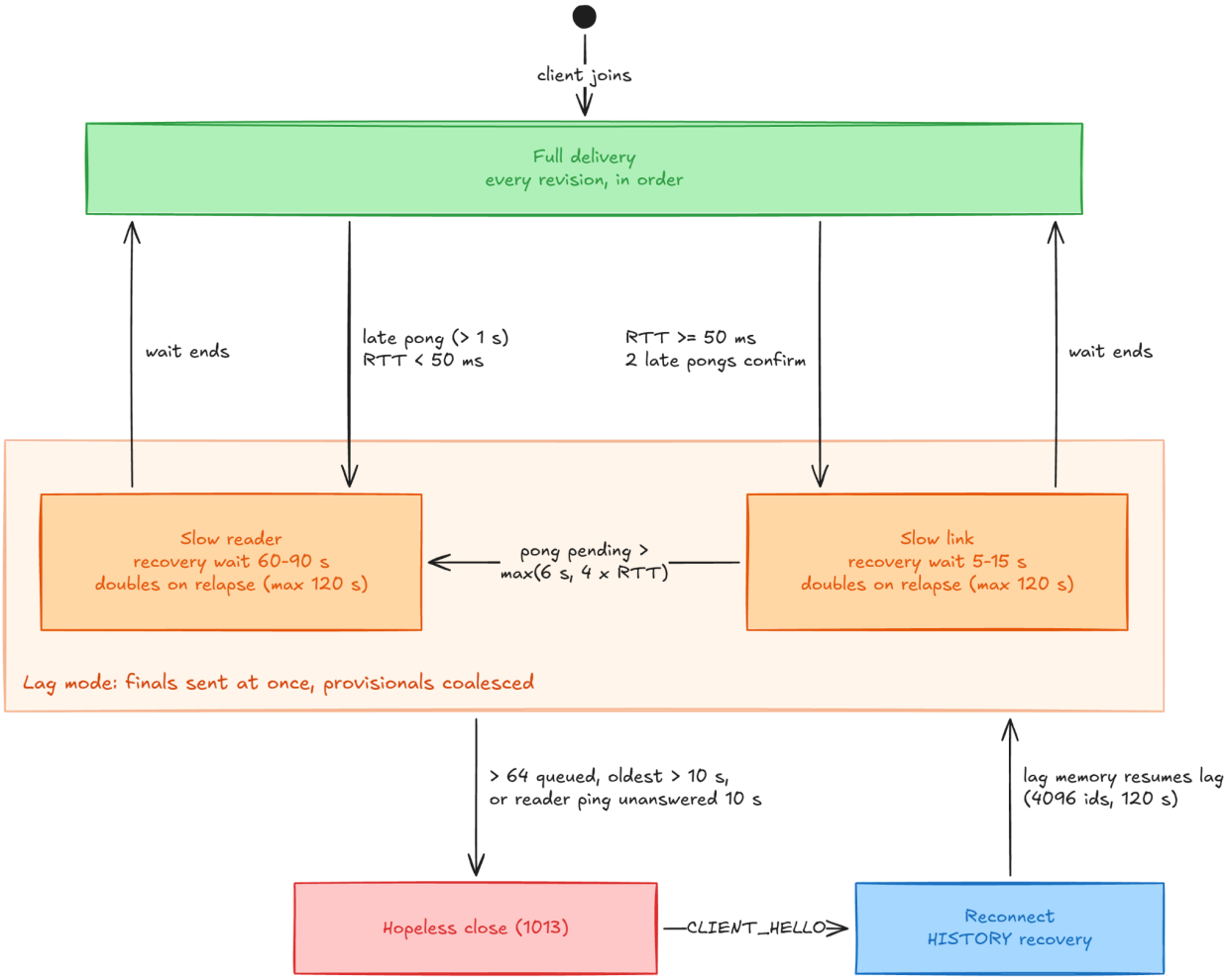
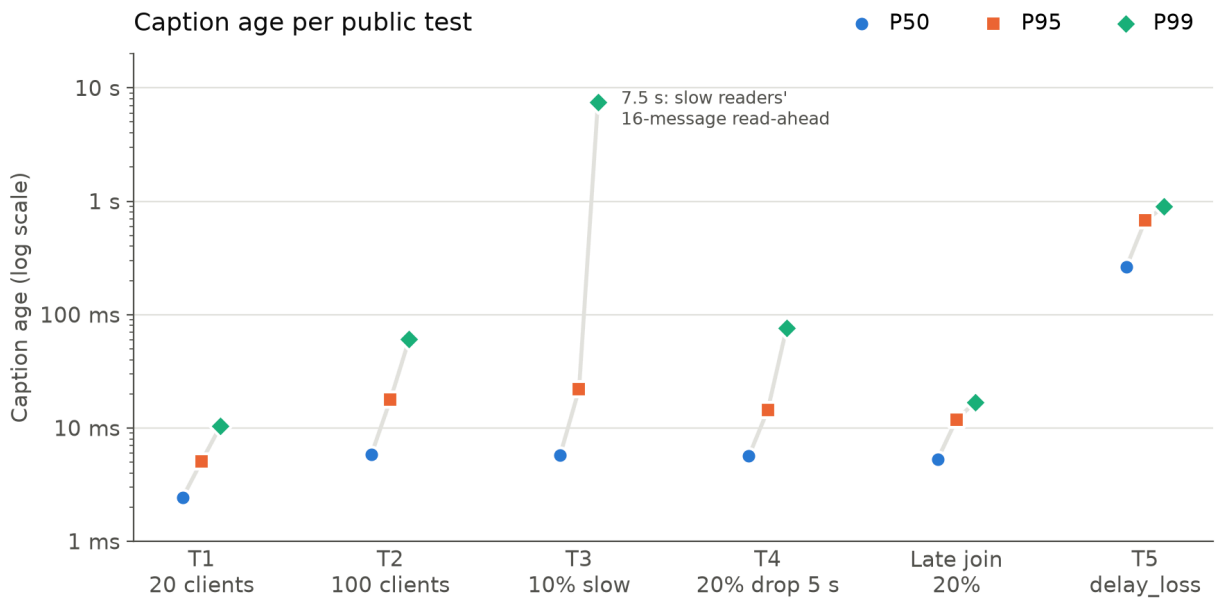


Figure 4: Caption age per public test

P50, P95 and P99 for T1 to T4, late join and T5 delay_loss on a log scale, since T3's P99 is in seconds and the rest in milliseconds (Section 6).



Ubuntu 26.04 VM (Apple Silicon, 2 vCPU), timer drift +0-1%, median of 3 runs

Figure 5: Caption age over time on 2 Mbit/s

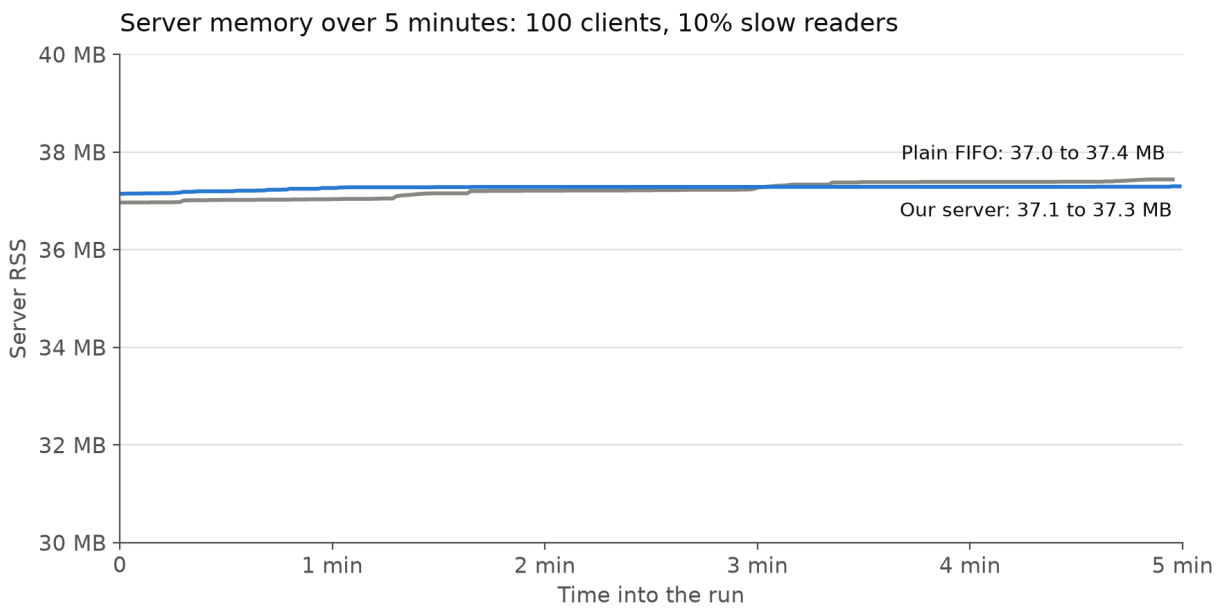
P50 and P95 caption age per 10 s window for the plain FIFO and our server (Section 7, Table B4).



Ubuntu 26.04 VM (Apple Silicon, 2 vCPU), timer drift +0-1%, median of 3 runs

Figure 6: Server memory over 5 minutes

Server RSS per second for both configurations, with the y-axis from 30 to 40 MB (Appendix B.4).



Ubuntu 26.04 VM (Apple Silicon, 2 vCPU), timer drift +0-1%, 1 run each

Appendix E: Evidence screenshots

E.1 Public tests gate

```
nic@Ubuntu:~/LiveRecallSystem$ cat results/final/check_full.txt
asyncio timer drift: +1%

### t1: T1 basic correctness, 20 clients
$ run_demo.py --clients 20 --duration 20 --runs 3 --port 8790 --output-stem results/check_t1

### t2: T2 public scale, 100 clients
$ run_demo.py --clients 100 --duration 40 --runs 3 --port 8790 --output-stem results/check_t2

### t3: T3 slow readers, 10% at 500 ms/msg
$ run_demo.py --clients 100 --duration 60 --slow-percent 10 --slow-ms 500 --runs 3 --port 8790 --output-stem results/check_t3

### t4: T4 reconnect, 20% disconnect for 5 s
$ run_demo.py --clients 100 --duration 60 --disconnect-percent 20 --disconnect-at 15 --disconnect-seconds 5 --runs 3 --port 8790 --output-stem results
/check_t4

### late: Late join, 20% of clients join after 8 s
$ run_demo.py --clients 100 --duration 40 --late-percent 20 --late-seconds 15 --runs 3 --port 8790 --output-stem results/check_late

scenario gate    final%   cover%    P50ms    P95ms    P99ms    dup  stale recovP95
t1#1     PASS     100.0    100.0      2.5      5.6      8.9      0      0      n/a
           server: RSS max 30.5 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t1#2     PASS     100.0    100.0      2.4      5.1     10.4      0      0      n/a
           server: RSS max 30.2 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t1#3     PASS     100.0    100.0      2.4      4.7     26.7      0      0      n/a
           server: RSS max 30.1 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t2#1     PASS     100.0    100.0      5.8     11.7     15.4      0      0      n/a
           server: RSS max 35.8 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t2#2     PASS     100.0    100.0      5.7     18.3     60.7      0      0      n/a
           server: RSS max 37.5 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t2#3     PASS     100.0    100.0      5.9     17.7    215.2      0      0      n/a
           server: RSS max 36.9 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t3#1     PASS     100.0     94.5      6.2     17.0   7505.8      0      0      n/a
           normal P95=12.8ms slow P95=9607.9ms slow fresh=25.8%
           server: RSS max 36.5 MB, lagging peak 10, max_pending 1, coalesced 1244, hopeless closes 0
t3#2     PASS     100.0     94.6      5.8     39.8   7524.2      0      0      n/a
           normal P95=12.6ms slow P95=9585.6ms slow fresh=25.0%
           server: RSS max 35.6 MB, lagging peak 10, max_pending 1, coalesced 1240, hopeless closes 0
t3#3     PASS     100.0     94.6      5.6     22.0   7546.5      0      0      n/a
           normal P95=12.1ms slow P95=9592.7ms slow fresh=25.0%
           server: RSS max 37.0 MB, lagging peak 10, max_pending 1, coalesced 1240, hopeless closes 0
t4#1     PASS     100.0     98.3      5.7     13.7     27.7      0      0      7.4
           server: RSS max 38.1 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t4#2     PASS     100.0     98.3      6.3     17.8     76.6      0      0      5.7
           server: RSS max 37.2 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
t4#3     PASS     100.0     98.3      5.1     14.4    106.4      0      0     31.0
           server: RSS max 38.5 MB, lagging peak 0, max_pending 1, coalesced 0, hopeless closes 0
late#1   PASS     100.0     92.3      5.1     11.5     28.3      0      0      2.2
           server: RSS max 36.9 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
late#2   PASS     100.0     92.2      6.3     12.0     14.0      0      0      2.6
           server: RSS max 37.5 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
late#3   PASS     100.0     92.3      5.3     11.9     16.9      0      0      3.3
           server: RSS max 36.2 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0

GATE PASSED
```

E.2 Constrained bandwidth

```
nic@Ubuntu:~/LiveRecallSystem$ cat results/final/bandwidth.txt
asyncio timer drift: +2%
$ sudo tc qdisc add dev lo root handle 1: prio bands 3 priomap 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
$ sudo tc qdisc add dev lo parent 1:1 handle 10: tbf rate 2mbit burst 4000 latency 400ms
$ sudo tc filter add dev lo parent 1: protocol ip prio 1 u32 match ip dport 8791 0xffff flowid 1:1
$ sudo tc filter add dev lo parent 1: protocol ip prio 1 u32 match ip sport 8791 0xffff flowid 1:1
qdisc on lo: qdisc tbf 10: parent 1:1 rate 2Mbit burst 4000b lat 400ms

### limited / before (plain FIFO (--no-coalesce)), run 1
$ LIVERECALL_NO_COALESCE=1 run_demo.py --clients 500 --duration 60.0 --runs 1 --port 8791 --output-stem results/bandwidth_limited_final_c500_before_ru
n01

### limited / after (lag-aware coalescing), run 1
$ run_demo.py --clients 500 --duration 60.0 --runs 1 --port 8791 --output-stem results/bandwidth_limited_final_c500_after_run01

### limited / before (plain FIFO (--no-coalesce)), run 2
$ LIVERECALL_NO_COALESCE=1 run_demo.py --clients 500 --duration 60.0 --runs 1 --port 8791 --output-stem results/bandwidth_limited_final_c500_before_ru
n02

### limited / after (lag-aware coalescing), run 2
$ run_demo.py --clients 500 --duration 60.0 --runs 1 --port 8791 --output-stem results/bandwidth_limited_final_c500_after_run02

### limited / before (plain FIFO (--no-coalesce)), run 3
$ LIVERECALL_NO_COALESCE=1 run_demo.py --clients 500 --duration 60.0 --runs 1 --port 8791 --output-stem results/bandwidth_limited_final_c500_before_ru
n03

### limited / after (lag-aware coalescing), run 3
$ run_demo.py --clients 500 --duration 60.0 --runs 1 --port 8791 --output-stem results/bandwidth_limited_final_c500_after_run03
```

```
Reset qdisc on lo

=== limited: 500 clients, 60 s, median of 3 runs, qdisc: qdisc tbf 10: parent 1:1 rate 2Mbit burst 4000b lat 400ms ===
| metric | before (plain FIFO (--no-coalesce)) | after (lag-aware coalescing) |
|---|---:|---:|
| final completeness % | 98.6 | 100.0 |
| normal update coverage % | 98.3 | 57.5 |
| P50 caption age ms | 1103 | 671 |
| P95 caption age ms | 1553 | 1244 |
| P99 caption age ms | 2438 | 1567 |
| fresh captions % (<=500 ms) | 0.0 | 31.3 |
| fresh finals % (<=500 ms) | 0.0 | 34.3 |
| duplicates | 0 | 0 |
| stale | 0 | 0 |
| connection errors | 2 | 0 |
| link Mbit/s, run average | 1.68 | 1.64 |
| link Mbit/s, peak 5 s | 1.78 | 1.77 |
| tbf packets dropped | 36974 | 10944 |
| lagging clients, peak | 0 | 399 |
| lag entries (slow link) | 0 | 994 |
| provisionals coalesced | 0 | 46088 |
| hopeless closes | 0 | 0 |
| max_pending, peak | 0 | 1 |
| server RSS MB, max | 65.5 | 67.3 |

written: results/bandwidth_limited_final_c500_summary.json
```

E.3 Stress

```
nic@Ubuntu:~/LiveRecallSystem$ cat results/final/stress.txt
asyncio timer drift: +0%

### s300: stress: 300 clients
$ run_demo.py --clients 300 --duration 40 --runs 3 --port 8790 --output-stem results/check_s300

### s500: stress: 500 clients
$ run_demo.py --clients 500 --duration 40 --runs 3 --port 8790 --output-stem results/check_s500

### s500slow: stress: 500 clients, 10% slow at 500 ms/msg
$ run_demo.py --clients 500 --duration 40 --slow-percent 10 --slow-ms 500 --runs 3 --port 8790 --output-stem results/check_s500slow

scenario gate    final%   cover%    P50ms    P95ms    P99ms    dup  stale recovP95
s300#1   PASS     100.0    100.0     14.8     26.2     29.8      0      0      n/a
           server: RSS max 51.4 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
s300#2   PASS     100.0    100.0     14.6     26.2     29.6      0      0      n/a
           server: RSS max 50.5 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
s300#3   PASS     100.0    100.0     14.6     26.7     30.7      0      0      n/a
           server: RSS max 51.9 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
s500#1   PASS     100.0    100.0     19.9     35.9    111.3      0      0      n/a
           server: RSS max 66.8 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
s500#2   PASS     100.0    100.0     19.7     34.3     90.1      0      0      n/a
           server: RSS max 65.9 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
s500#3   PASS     100.0    100.0     18.9     32.6     36.5      0      0      n/a
           server: RSS max 65.9 MB, lagging peak 0, max_pending 0, coalesced 0, hopeless closes 0
s500slow#1 PASS     100.0     95.1     18.1    789.4   7851.0      0      0      n/a
           normal P95=31.3ms slow P95=9113.5ms slow fresh=2.7%
           server: RSS max 66.9 MB, lagging peak 50, max_pending 1, coalesced 3600, hopeless closes 0
s500slow#2 PASS     100.0     95.1     18.3    876.5   7950.8      0      0      n/a
           normal P95=34.3ms slow P95=9111.7ms slow fresh=2.7%
           server: RSS max 66.2 MB, lagging peak 50, max_pending 1, coalesced 3600, hopeless closes 0
s500slow#3 PASS     100.0     95.1     18.0    834.8   7869.6      0      0      n/a
           normal P95=30.9ms slow P95=9125.1ms slow fresh=2.7%
           server: RSS max 66.0 MB, lagging peak 50, max_pending 1, coalesced 3600, hopeless closes 0

GATE PASSED
```

E.4 Overload policy code

The overload policy is implemented in liverecall/channel.py, mainly in offer() and _check_lag().

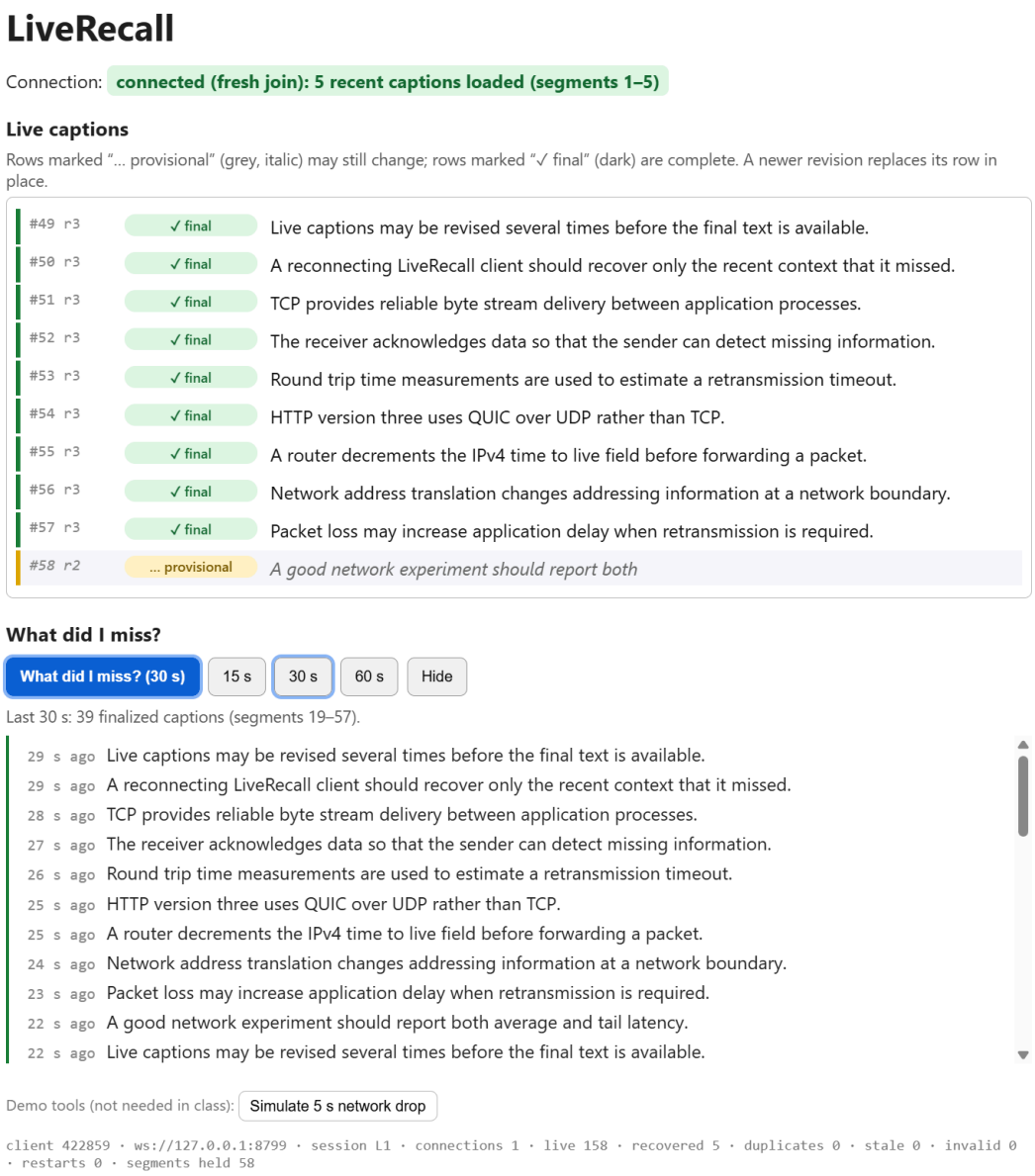
```python
    def offer(self, caption: Caption) -> None:
        """Queue a caption update. Must not await (called from the source loop)."""
        if self.closed:
            return
        if self.coalescing:
            entry = self._latest.get(caption.segment)
            if entry is not None and entry.caption is not None and not entry.caption.final:
                entry.wire = caption.wire
                entry.caption = caption
                self.coalesced += 1
                self._wake()
                self._check_hopeless()
                return
        entry = Outgoing(caption.wire, caption)
        self._queue.append(entry)
        self._latest[caption.segment] = entry
        if len(self._queue) > self.max_pending:
            self._compact()
            if len(self._queue) > self.max_pending:
                self._overflow()
                return
        self._wake()
        self._check_hopeless()
```

```
464     def _check_lag(self) -> None:
465         pong = self._pong
466         if pong is None or pong.done():
467             return
468         now = asyncio.get_running_loop().time()
469         if now - self._ping_at <= self.lag_s:
470             return
471         if self._pong_hopeless():
472             self._overflow()
473             return
474         if self.lagging:
475             self._calm_since = None  # an overdue pong is not calm
476             if self.link_lag and now - self._ping_at > self._reader_pong_s():
477                 self._become_reader_lag()
478             return
479         link = self._slow_link()
480         if link and not self._link_confirmed(now):
481             return
482         self._strikes = 0
483         self.link_lag = link
484         self.link_lag_entries += self.link_lag
485         base = self._link_recover_s if self.link_lag else self._base_recover_s
486         if self._left_lag_at is not None and now - self._left_lag_at < REARM_S:
487             # Relapse: double, but never below this cause's base (a link relapse
488             # followed by a reader stall must not retry the reader after 20 s).
489             self._recover_s = max(base, min(self._recover_s * 2, RECOVER_MAX_S))
490         else:
491             self._recover_s = base
492         # A slow reader's wait is a floor (an early retry refills its read-ahead and
493         # loses tail finals: Day 4 part 2, T3 60 s 99.83%), so jitter only upwards.
494         low = 1 - self.jitter if self.link_lag else 1.0
495         self._recover_wait = self._recover_s * random.uniform(low, 1 + self.jitter)
496         self._calm_since = None
497         self.lagging = True
498         self.coalescing = True
499         self.lag_entries += 1
500         self._compact()
```

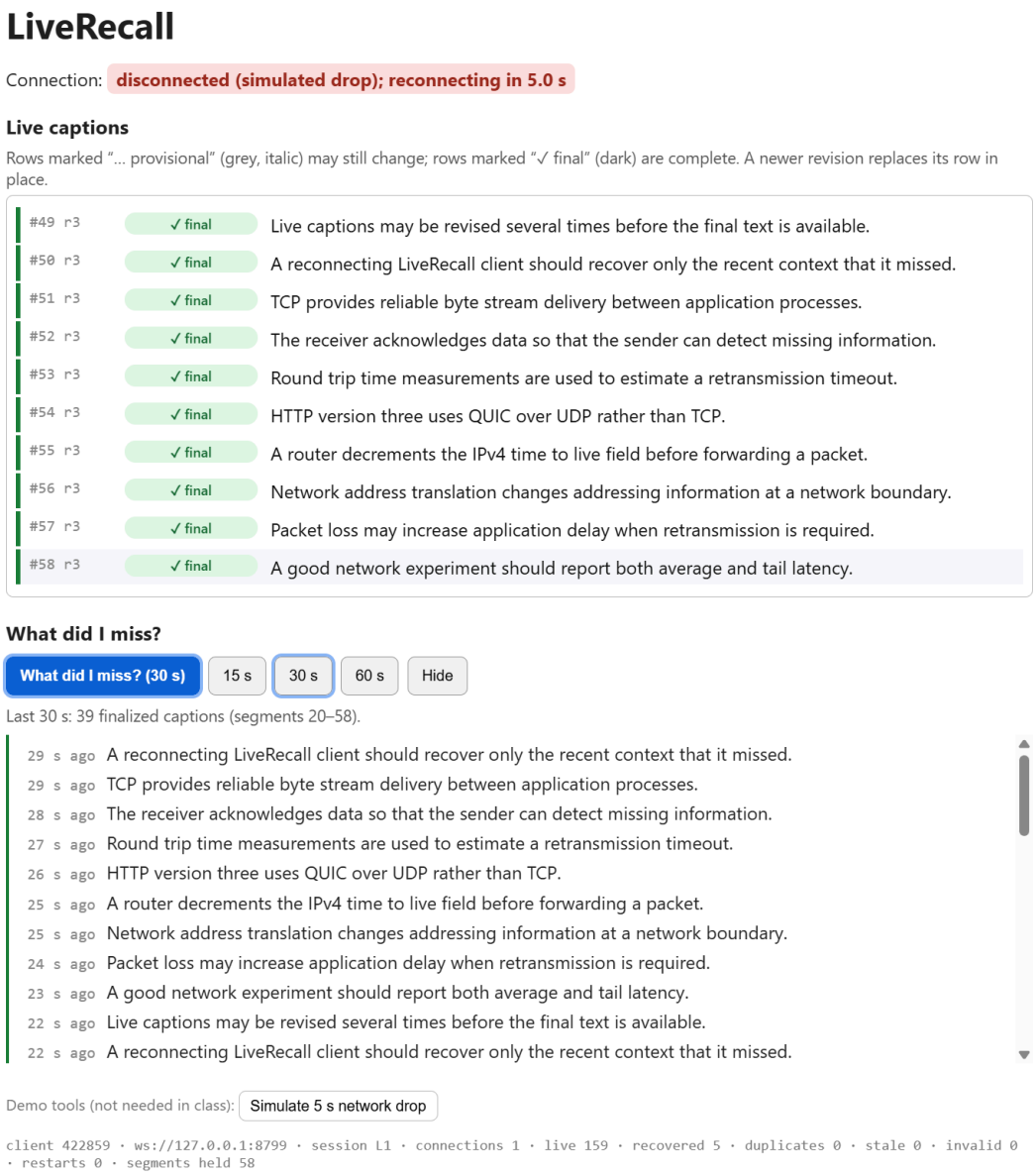
E.5 Browser client

Browser client against the submitted server on a Windows laptop.

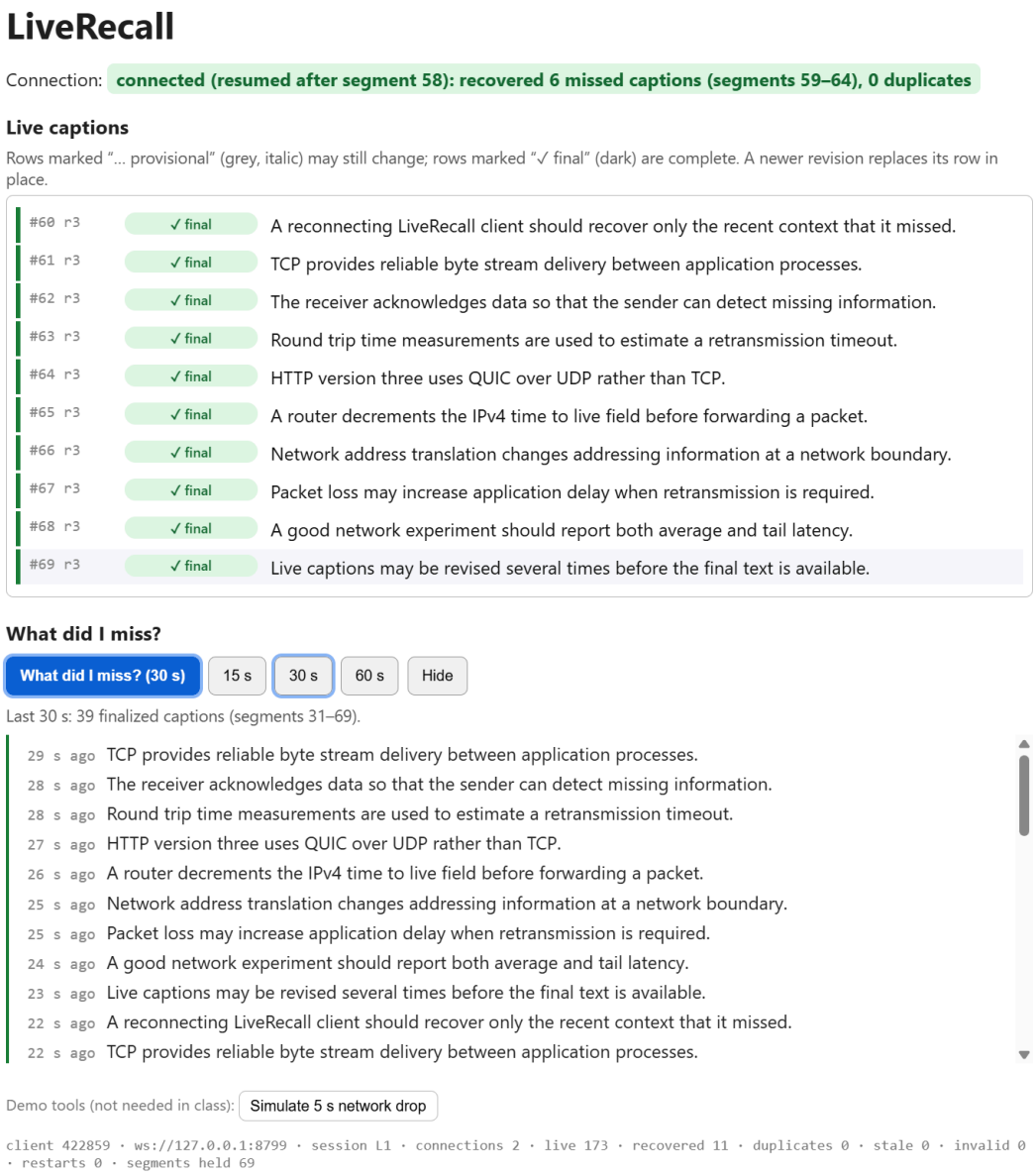
(a) provisional and final rows with the 30 s recall panel



(b) during a simulated 5 s drop



(c) after resuming with 6 finals recovered from HISTORY and 0 duplicates



Appendix F: Video Link - Screen Recording 2026-11-03 at 3.02.34 AM.mov